УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 11

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ РАБОТЫ С БИНАРНЫМИ ФАЙЛАМИ И ФАЙЛАМИ EXCEL

ЛАБОРАТОРНАЯ РАБОТА № 11.2

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ РАБОТЫ С ФАЙЛАМИ EXCEL

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ для работы с файлами Excel

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Файлы CSV

Одним из распространенных файловых форматов, которые хранят в удобном виде информацию, является формат **csv**. Каждая строка в файле csv представляет отдельную запись или строку, которая состоит из отдельных столбцов, разделенных запятыми. Собственно, поэтому формат и называется Comma Separated Values. Но хотя формат csv — это формат текстовых файлов, Python для упрощения работы с ним предоставляет специальный встроенный модуль **csv**.

Рассмотрим работу модуля на примере:

```
import csv
FILENAME = "users.csv"
users = [
    ["Tom", 28],
    ["Alice", 23],
    ["Bob", 34]
]
with open(FILENAME, "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(users)
with open(FILENAME, "a", newline="") as file:
    user = ["Sam", 31]
    writer = csv.writer(file)
    writer.writerow(user)
```

В файл записывается двухмерный список - фактически таблица, где каждая строка представляет одного пользователя. А каждый пользователь содержит два поля - имя и возраст. То есть фактически таблица из трех строк и двух столбцов.

При открытии файла на запись в качестве третьего параметра указывается значение newline="" - пустая строка позволяет корректно считывать строки из файла вне зависимости от операционной системы.

Для записи нам надо получить объект **writer**, который возвращается функцией csv.writer(file). В эту функцию передается открытый файл. А собственно запись производится с помощью метода writer.writerows(users) Этот метод принимает набор строк. В нашем случае это двухмерный список.

Если необходимо добавить одну запись, которая представляет собой одномерный список, например, ["Sam", 31], то в этом случае можно вызвать метод **writer.writerow(user)**

В итоге после выполнения скрипта в той же папке окажется файл users.csv, который будет иметь следующее содержимое:

```
Tom,28
Alice,23
Bob,34
Sam,31
```

Для чтения из файла нам наоборот нужно создать объект **reader**:

```
import csv
FILENAME = "users.csv"
with open(FILENAME, "r", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row[0], " - ", row[1])
```

При получении объекта reader мы можем в цикле перебрать все его строки:

```
Tom  -  28
Alice  -  23
Bob  -  34
Sam – 31
```

4.2 Работа со словарями

В примере выше каждая запись или строка представляла собой отдельный список, например, ["Sam", 31]. Но кроме того, модуль csv имеет специальные дополнительные возможности для работы со словарями. В частности, функция **csv.DictWriter()** возвращает объект writer, который позволяет записывать в файл. А функция **csv.DictReader()** возвращает объект reader для чтения из файла. Например:

```
import csv
FILENAME = "users2.csv"
users = [
    {"name": "Tom", "age": 28},
    {"name": "Alice", "age": 23},
    {"name": "Bob", "age": 34}
]
with open(FILENAME, "w", newline="") as file:
```

```
    columns = ["name", "age"]
    writer = csv.DictWriter(file, fieldnames=columns)
    writer.writeheader()
    # запись нескольких строк
    writer.writerows(users)
    user = {"name" : "Sam", "age": 41}
    # запись одной строки
    writer.writerow(user)
with open(FILENAME, "r", newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["name"], "-", row["age"])
```

Запись строк также производится с помощью методов writerow() и writerows(). Но теперь каждая строка представляет собой отдельный словарь, и кроме того, производится запись и заголовков столбцов с помощью метода **writeheader()**, а в метод csv.DictWriter в качестве второго параметра передается набор столбцов.

При чтении строк, используя названия столбцов, мы можем обратиться к отдельным значениям внутри строки: row["name"].

4.3 Установка OpenPyxl

Для установки библиотеки **OpenPyxl** в **Python** введите в командной строке (Windows) или терминале (macOS, Linux или в самой среде разработки) команду:

pip install openpyxl

Виртуальное окружение — это изолированная среда для работы с Python-проектами. Это позволяет избежать конфликтов зависимостей между проектами.

Виртуальное окружение можно создать с помощью команды:

python -m venv myenv

Далее нужно активировать окружение:

Windows

.\myenv\Scripts\activate

Linux

source myenv/bin/activate

Установите **OpenPyxl** в виртуальном окружении с помощью команды установки.

После установки **OpenPyxl** проверьте, что все работает.

Запустите **Python** в терминале:

Это откроет интерактивный режим **Python**.

Внутри **Python** (терминале или консоли) выполните команды:

```
import openpyxl
print(openpyxl.__version__)
```

Если библиотека установлена правильно, отобразится номер версии **OpenPyxl**.

4.4 Чем полезен OpenPyxl

При работе с **OpenPyxl** мы не запускаем **Excel**, а создаем и манипулируем файлами **Excel** (.xlsx) через **Python**. После создания файла можно открыть его в **Excel**, но сам процесс манипуляции файлами выполняется без него.

Перед работой с **OpenPyxl** в среде разработки нужно импортировать библиотеку:

```
import openpyxl
```

4.5 Работа с файлами

Чтобы создать новый Excel-файл, нужно сначала создать рабочую книгу (**Workbook**), а затем добавлять листы и записывать данные в ячейки.

Workbook в библиотеке **openpyxl** — это класс, который используется для создания нового Excel-файла.

Для работы с **Workbook**, нужно импортировать специальные библиотеки.

Перед созданием нового файла импортируйте класс Workbook:

```
from openpyxl import Workbook
```

Для работы с существующим файлом нужно использовать load_workbook^

```
from openpyxl import load_workbook
```

Можно обойтись и импортом библиотеки **openpyxl**, но тогда перед функциями придется писать **openpyxl**. Вместо **load_workbook** будет **openpyxl.load_workbook**.

Пример:

```
from openpyxl import Workbook
# Создаем новую рабочую книгу (файл)
wb = openpyxl.Workbook()
# Получаем активный лист (по умолчанию создается один лист)
sheet = wb.active
# Устанавливаем имя листа
sheet.title = "Лист1"
# Записываем данные в ячейки
sheet['A1'] = "Привет"
sheet['A2'] = "Мир"
# Сохраняем файл на диск
wb.save('новый_файл.xlsx')
```

В методе **save()** можно указать либо только имя файла, либо полный путь к нему.

Если указано только имя файла, как в примере («новый_файл.xlsx»), файл будет сохранен в текущей рабочей директории программы (папке, из которой запускается скрипт).

Если нужно сохранить файл в определенной папке, то в кавычках стоит указать полный путь к файлу, например:

```
wb.save(r"C:\Users\User\Documents\новый_файл.xlsx")
```

*Символ **r** перед строкой пути используют, чтобы избежать проблем с символами обратного слэша (\).*

Итак, мы создали новый файл **new_file.xlsx**, добавили лист с данными в ячейки **A1** и **A2** и сохранили файл.

Для открытия существующего файла используется метод **load_workbook()**.

Пример:

```
wb = load_workbook('file.xlsx')
```

Эта строка откроет файл **file.xlsx**.

4.6 Работа с листами

Активный лист — это тот, который открыт по умолчанию, когда вы загружаете или создаете файл. Получить его можно с помощью метода **active**.

Пример:

```
from openpyxl import load_workbook
# Открываем существующий файл
wb = load_workbook('file.xlsx')
# Получаем активный лист
sheet = wb.active
# Выводим название активного листа
print(f"Активный лист: {sheet.title}")
```

Можно создать новый лист с определенным именем при помощи функции **create_sheet()**.

Пример:

```
from openpyxl import Workbook
# Создаем новую книгу
wb = Workbook()
# Или загружаем готовую
wb = load_workbook('file.xlsx')
# Добавляем новый лист
new_sheet = wb.create_sheet(title="Новый Лист")
# Сохраняем изменения
wb.save("file_with_new_sheet.xlsx")
```

Чтобы изменить название листа, просто присвойте новое имя в его свойство **title**.

Пример:

```
# Открываем файл
wb = load_workbook("file_with_new_sheet.xlsx")
# Получаем лист по имени
sheet = wb["Новый Лист"]
# Меняем название листа
sheet.title = "Переименованный Лист"
# Сохраняем изменения
wb.save("renamed_sheet.xlsx")
```

Удалить лист можно с помощью метода **remove()**.

Пример:

```
# Открываем файл
wb = load_workbook("renamed_sheet.xlsx")
# Получаем лист для удаления
sheet_to_remove = wb["Переименованный Лист"]
# Удаляем лист
wb.remove(sheet_to_remove)
# Сохраняем изменения
wb.save("sheet_removed.xlsx")
```

4.7 Работа с данными

Вы можете получить значение из конкретной ячейки, указав ее координаты (например, **A1**).

Пример:

```
from openpyxl import load_workbook
# Открываем существующий файл
wb = load_workbook("file.xlsx")
# Получаем активный лист
sheet = wb.active
# Читаем значение ячейки A1
value = sheet["A1"].value
print(f"Значение ячейки A1: {value}")
```

Для чтения значений нескольких ячеек можно использовать перебор по диапазону.

Пример:

```
from openpyxl import load_workbook
# Открываем файл
wb = load_workbook("file.xlsx")
sheet = wb.active
# Перебираем ячейки в диапазоне A1:C3
for row in sheet["A1:C3"]: # Указываем диапазон
for cell in row:
print(f"Ячейка {cell.coordinate}: {cell.value}")
```

Для записи данных в ячейки используйте объект листа и укажите координаты ячейки.

Пример:

```
from openpyxl import Workbook
# Создаем новую книгу
wb = Workbook()
sheet = wb.active
# Записываем текст и числа
sheet["A1"] = "Текст" # Записываем текст
sheet["B1"] = 123 # Записываем число
```

```
sheet["C1"] = 45.67 # Записываем дробное число
# Сохраняем файл
wb.save("file_with_data.xlsx")
```

Вы можете задать ширину столбцов и высоту строк для улучшения читаемости данных.

Пример:

```
# Устанавливаем ширину столбца A = 20 и высоту строки 1 = 30
sheet.column_dimensions["A"].width = 20
sheet.row_dimensions[1].height = 30
```

Для форматирования используйте модуль **openpyxl.styles**.

Пример:

```
from openpyxl.styles import Font, Alignment
# Применяем жирный шрифт
sheet["A1"].font = Font(bold=True)
# Изменяем цвет текста (например, синий)
sheet["B1"].font = Font(color="0000FF")
# Выравнивание текста по центру
sheet["C1"].alignment = Alignment(horizontal="center", vertical="center")
```

4.8 Работа с таблицами и формулами

Для создания таблицы данных используйте объект **Table** из модуля **openpyxl.worksheet.table**.

Пример:

```
from openpyxl import Workbook
from openpyxl.worksheet.table import Table, TableStyleInfo
# Создаем новую книгу и активный лист
wb = Workbook()
sheet = wb.active
# Заполняем данные
data = [
["Имя", "Возраст", "Город"],
["Иван", 25, "Москва"],
["Анна", 30, "Санкт-Петербург"],
["Петр", 35, "Новосибирск"]
]
for row in data:
sheet.append(row)
# Определяем диапазон для таблицы
table = Table(displayName="MyTable", ref="A1:C4")
# Настраиваем стиль таблицы
style = TableStyleInfo(
name="TableStyleMedium9", showFirstColumn=False,
showLastColumn=False, showRowStripes=True, showColumnStripes=True
)
```

```
table.tableStyleInfo = style
# Добавляем таблицу на лист
sheet.add_table(table)
# Сохраняем файл
wb.save("file_with_table.xlsx")
```

Для работы с формулами просто задайте в ячейке формулу как строку (например, =SUM(A1:A10)).

Пример:

```
from openpyxl import Workbook
# Создаем новую книгу
wb = Workbook()
sheet = wb.active
# Заполняем данные
sheet["A1"] = 10
sheet["A2"] = 20
sheet["A3"] = 30
# Добавляем формулу
sheet["A4"] = "=SUM(A1:A3)" # Сумма ячеек A1, A2 и A3
# Сохраняем файл
wb.save("file_with_formula.xlsx")
```

4.9 Настройка высоты строк и ширины столбцов

Объекты Worksheet имеют атрибуты row_dimensions и column_dimensions, которые управляют высотой строк и шириной столбцов.

```
sheet['A1'] = 'Высокая строка'
sheet['B2'] = 'Широкий столбец'
sheet.row_dimensions[1].height = 70
sheet.column_dimensions['B'].width = 30
```

Атрибуты row_dimensions и column_dimensions представляют собой значения, подобные словарю. Атрибут row_dimensions содержит объекты RowDimensions, а атрибут column_dimensions содержит объекты ColumnDimensions. Доступ к объектам в row_dimensions осуществляется с использованием номера строки, а доступ к объектам в column_dimensions — с использованием буквы столбца.

Для указания высоты строки разрешено использовать целые или вещественные числа в диапазоне от 0 до 409. Для указания ширины столбца можно использовать целые или вещественные числа в диапазоне от 0 до 255. Столбцы с нулевой шириной и строки с нулевой высотой невидимы для пользователя.

4.10 Объединение ячеек

Ячейки, занимающие прямоугольную область, могут быть объединены в одну ячейку с помощью метода merge_cells() рабочего листа:

```
sheet.merge_cells('A1:D3')
```

```
sheet['A1'] = 'Объединены двенадцать ячеек'
sheet.merge_cells('C5:E5')
sheet['C5'] = 'Объединены три ячейки'
```

Чтобы отменить слияние ячеек, надо вызвать метод unmerge_cells():

```
sheet.unmerge_cells('A1:D3')
sheet.unmerge_cells('C5:E5')
```

4.11 Закрепление областей

Если размер таблицы настолько велик, что ее нельзя увидеть целиком, можно заблокировать несколько верхних строк или крайних слева столбцов в их позициях на экране. В этом случае пользователь всегда будет видеть заблокированные заголовки столбцов или строк, даже если он прокручивает таблицу на экране.

У объекта Worksheet имеется атрибут freeze_panes, значением которого может служить объект Cell или строка с координатами ячеек. Все строки и столбцы, расположенные выше и левее, будут заблокированы.

4.12 Диаграммы

Модуль OpenPyXL поддерживает создание гистограмм, графиков, а также точечных и круговых диаграмм с использование данных, хранящихся в электронной таблице. Чтобы создать диаграмму, необходимо выполнить следующие действия:

– создать объект Reference на основе ячеек в пределах выделенной прямоугольной области;

– создать объект Series, передав функции Series() объект Reference;

– создать объект Chart;

– дополнительно можно установить значения переменных drawing.top, drawing.left, drawing.width, drawing.height объекта Chart, определяющих положение и размеры диаграммы;

– добавить объект Chart в объект Worksheet.

Объекты Reference создаются путем вызова функции openpyxl.charts.Reference(), принимающей пять аргуменов:

– объект Worksheet, содержащий данные диаграммы;

– два целых числа, представляющих верхнюю левую ячейку выделенной прямоугольной области, в которых содержатся данные диаграммы: первое число задает строку, второе — столбец; первой строке соответствует 1, а не 0;

– два целых числа, представляющих нижнюю правую ячейку выделенной прямоугольной области, в которых содержатся данные диаграммы: первое число задает строку, второе — столбец.

```
from openpyxl import Workbook
from openpyxl.chart import BarChart, Reference
# создаем новый excel-файл
```

```
wb = Workbook()
# добавляем новый лист
wb.create_sheet(title = 'Первый лист', index = 0)
# получаем лист, с которым будем работать
sheet = wb['Первый лист']
sheet['A1'] = 'Серия 1'
# это колонка с данными
for i in range(1, 11):
    cell = sheet.cell(row = i + 1, column = 1)
    cell.value = i * i
# создаем диаграмму
chart = BarChart()
chart.title = 'Первая серия данных'
data = Reference(sheet, min_col = 1, min_row = 1, max_col = 1, max_row = 11)
chart.add_data(data, titles_from_data = True)
# добавляем диаграмму на лист
sheet.add_chart(chart, 'C2')
# записываем файл
wb.save('example.xlsx')
```

Результат выполнения представлен на рисунке 1:

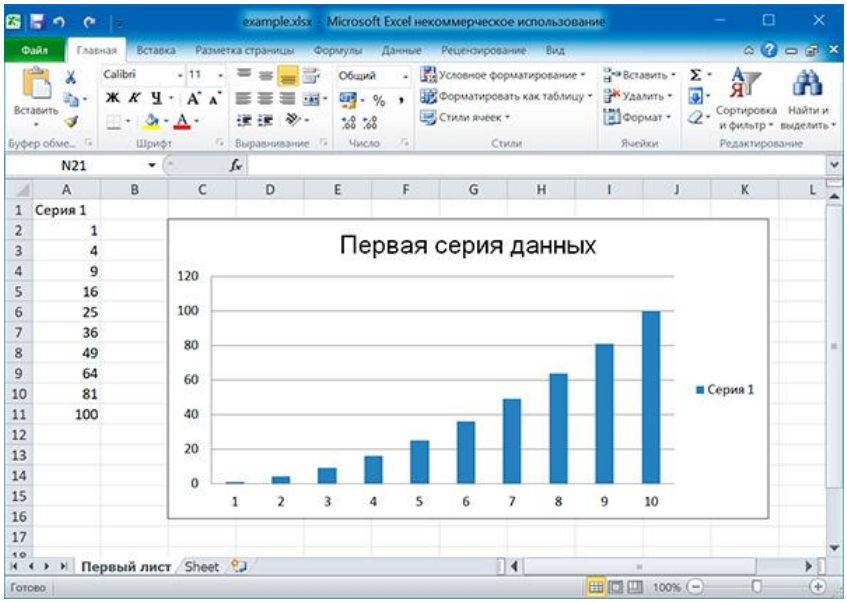


Рисунок 1 - Результат выполнения диаграммы

Аналогично можно создавать графики, точечные и круговые диаграммы, вызывая методы:

- openpyxl.chart.LineChart();
- openpyxl.chart.ScatterChart();
- openpyxl.chart.PieChart().

4.7 Пример решения задачи на языке программирования Python

Задача. Сгенерировать отчет с динамическими данными.

Решение задачи:

1 **Создаем новую книгу и активный лист**

Используем **Workbook** для создания нового файла **Excel**. Активный лист выбирается автоматически с помощью **sheet = wb.active**.

```
from openpyxl import Workbook
from openpyxl.styles import Font
wb = Workbook()
sheet = wb.active
sheet.title = "Отчет"
```

2 **Добавляем заголовок отчета**

Записываем текст в ячейку **A1**. Делаем шрифт жирным и увеличиваем размер с помощью **Font**.

```
sheet["A1"] = "Отчет по продажам"
sheet["A1"].font = Font(bold=True, size=14)
```

3 **Создаем таблицу с данными**

Вручную заполняем строки. Используем формулы Excel для расчета столбца **«Сумма»**.

```
data = [
["Товар", "Количество", "Цена", "Сумма"],
["Телефон", 10, 20000, "=B2*C2"],
["Ноутбук", 5, 50000, "=B3*C3"],
["Планшет", 7, 15000, "=B4*C4"]
]
for row in data:
sheet.append(row)
```

4 **Добавляем итоговую строку**

Записываем формулу в ячейку **D6** для расчета суммы столбца **«Сумма»**. Стилизуем ячейку **«Итого»**.

```
sheet["D6"] = "=SUM(D2:D4)"
sheet["C6"] = "Итого:"
sheet["C6"].font = Font(bold=True)
```

5 **Сохраняем отчет в файл**

```
wb.save("sales_report.xlsx")
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте Excel-файл с 2 листами: «Продажи» и «Сотрудники», на листе «Продажи» заполните случайные данные о товарах, количестве и цене.

2 Создайте Excel-файл и добавьте вычисляемый столбец «Сумма», где рассчитывается произведение «Количество × Цена».

3 Создайте Excel-файл и вычислите среднее значение столбца «Сумма».

4 Создайте Excel-файл и отсортируйте данные по убыванию значений столбца «Сумма», затем сохраните результат.

5 Создайте Excel-файл и выделите цветом ячейки, в которых значение «Сумма» превышает 1000.

6 Создайте Excel-файл, в котором один столбец содержит случайные даты текущего месяца, а второй — значения продаж.

7 Создайте Excel-файл и определите количество продаж, совершённых в выходные дни (суббота и воскресенье).

8 Создайте Excel-файл с таблицей оценок студентов и добавьте столбец «Статус», где указано «Сдал» (≥ 50) и «Не сдал» (< 50).

9 Создайте Excel-файл с 10 случайными числами от 1 до 100 и выделите минимальное и максимальное значения.

10 Создайте Excel-файл и сгенерируйте таблицу умножения размером N×N, где N вводится пользователем.

11 Создайте Excel-файл и преобразуйте все текстовые данные в верхний регистр.

12 Создайте Excel-файл с расписанием на неделю (дни — строки, часы — столбцы), заполните его случайными задачами.

13 Создайте Excel-файл и найдите максимальное и минимальное значение в числовом столбце.

14 Создайте Excel-файл и удалите все пустые строки из таблицы.

15 Создайте Excel-файл и замените все отрицательные числа на нули.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте Excel-файл с данными о продажах за 12 месяцев и постройте линейную диаграмму, отображающую динамику продаж.

2 Создайте Excel-файл с таблицей «Месяц — Количество клиентов» и постройте столбчатую диаграмму (ColumnChart).

3 Создайте Excel-файл и постройте круговую диаграмму, показывающую процентное распределение продаж трёх товаров.

4 Создайте Excel-файл с результатами тестирования студентов (10 человек) и постройте гистограмму (BarChart), отображающую их оценки.

5 Создайте Excel-файл с данными о доходах фирмы в различных городах и постройте диаграмму с накоплением (Stacked Bar).

6 Создайте Excel-файл с данными «Год — Количество аварий на дорогах» и постройте точечную диаграмму (ScatterChart).

7 Создайте Excel-файл и постройте диаграмму с областями (AreaChart), отображающую рост населения города за 15 лет.

8 Создайте Excel-файл с расходами по категориям (еда, транспорт, коммунальные услуги, развлечения, медицина) и постройте круговую диаграмму.

9 Создайте Excel-файл с данными о температуре воздуха по дням месяца и постройте линейную диаграмму (LineChart).

10 Создайте Excel-файл с затратами трёх отделов фирмы по месяцам и постройте составную столбчатую диаграмму (Stacked Column).

11 Создайте Excel-файл и постройте диаграмму с подписями значений для каждого столбца.

12 Создайте Excel-файл с количеством шагов за неделю и постройте линейную диаграмму, выделив день с максимальным значением.

13 Создайте Excel-файл с данными о времени работы за компьютером за 30 дней и постройте точечную диаграмму (ScatterChart).

14 Создайте Excel-файл с данными о продажах пяти товаров и постройте комбинированную диаграмму (Line + Column).

15 Создайте Excel-файл с данными о посещаемости сайта по дням недели и постройте диаграмму, сравнивающую значения по дням.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Какие библиотеки используются в языке программирования Python для работы с файлами Excel?

7.2 В чём отличие библиотек openpyxl и pandas при работе с Excel-файлами в языке программирования Python?

7.3 Как создать новый Excel-файл с помощью openpyxl в языке программирования Python?

7.4 Что такое Workbook и Worksheet в языке программирования Python?

7.5 Каким образом можно получить доступ к ячейке A1 в openpyxl в языке программирования Python?

7.6 Как записать значение в ячейку Excel-файла в языке программирования Python?

7.7 Как сохранить файл Excel после изменения данных в языке программирования Python?

7.8 Какие режимы чтения и записи поддерживает openpyxl в языке программирования Python?

7.9 Как получить количество строк и столбцов листа в языке программирования Python?

7.10 Каким методом можно добавить новую строку в лист в языке программирования Python?

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.